

```
# sage aes26.sage
Plaintext: Open Source For you
Encrypted Text: P4ILOy1/rqWo6koTrKlFy2EC3Jcf8rTh/ndLTlnKKD9DlpIcWaA6Zp++zmjlvLjc
Decrypted Text: Open Source For you
```

Figure 2: Execution of the SageMath program *aes26.sage*

```
# sage stream27.sage
Plaintext: HAPPINESSISABUTTERFLY
Ciphertext: f1ff9206518a58e1410c111f2290cb5b41fef72eb16c58ca0e04da9b43853c82dcfe2fcaecbf8bfe
Decrypted Text: HAPPINESSISABUTTERFLY
```

Figure 3: Execution of the SageMath program *stream27.sage*

Stream ciphers

The story of symmetric-key cryptography does not end with block ciphers alone; stream ciphers must also be considered. A stream cipher is a type of symmetric encryption algorithm that encrypts data one bit or byte at a time, rather than processing it in fixed-size blocks, as in block ciphers like DES, Triple DES, and AES. This makes stream ciphers fast, efficient, and ideal for applications requiring real-time encryption. However, stream ciphers have a vulnerability: if the same keystream is used to encrypt multiple messages, an attacker can recover the plaintext. ChaCha20, Salsa20, XSalsa20, Grain, and RC4 are examples of stream ciphers.

Now, consider the SageMath program named ‘stream27.sage’ shown below which implements XSalsa20 stream cipher. XSalsa20 is a stream cipher that extends the Salsa20 stream cipher by using a 192-bit nonce instead of a 64-bit nonce, making it more secure against nonce reuse attacks. A nonce (number used once) is a unique, randomly chosen or sequentially incremented value used to ensure that encryption is non-deterministic. This prevents attackers from detecting patterns in encrypted messages. XSalsa20 and Salsa20 were designed by Daniel J. Bernstein and are part of the ‘NaCl’ (Networking and Cryptography Library). The program ‘stream27.sage’ uses the library ‘PyNaCl’ for secret-key encryption and ‘PyCryptodome’ for padding and unpadding plaintext (line numbers are included for clarity).

```
1. import nacl.secret
2. import nacl.utils
3. from Crypto.Util.Padding import pad, unpad
4. key = (b"RAINBOWS" * 4)[:32]
5. plaintext = "HAPPINESSISABUTTERFLY"
6. print("Plaintext:", plaintext)
7. box = nacl.secret.SecretBox(key)
8. plaintext_padded = pad(plaintext.encode( ), 8)
9. ciphertext = box.encrypt(plaintext_padded)
10. print("Ciphertext (hex):", ciphertext.ciphertext.hex( ))
11. decrypted_padded = box.decrypt(ciphertext)
12. decrypted_message = unpad(decrypted_padded, 8).decode( )
13. print("Decrypted Text:", decrypted_message)
```

Now, let us go through the code line by line to understand how it works. Line 1 imports *nacl.secret*, which provides secret-key encryption using the XSalsa20 encryption algorithm with Poly1305 authentication. Line 2 imports *nacl.utils*, which contains utility functions like random key generation. Line 3 imports the functions *pad()*

and *unpad()* from the module *Crypto.Util.Padding* offered by ‘PyCryptodome’. In Line 4, the encryption key is generated from the string “RAINBOWS”, which is repeated 4 times to form *b“RAINBOWSRainbowsRainbowsRainbows”*. The index *[:32]* ensures that only the first 32 bytes are used (as required by *nacl.secret.SecretBox*, which expects a 32-byte key). Line 5 defines the plaintext message “HAPPINESSISABUTTERFLY”, which will be encrypted. Line 6 prints the plaintext before encryption. Line 7 initialises an instance named *box* of *SecretBox* using the key generated. In Line 8, the function *encode()* converts the plaintext string into bytes and the function *pad()* applies PKCS7 padding to make the plaintext’s length a multiple of 8 bytes. Line 9 encrypts the padded plaintext using XSalsa20 encryption and authenticates with Poly1305 authentication. Line 10 extracts and prints the ciphertext in hexadecimal format. Line 11 decrypts ciphertext using the instance *box* of *SecretBox*. Line 12 removes the PKCS7 padding using the function *unpad()* and converts the bytes back to a string. Finally, Line 13 prints the decrypted message. Upon execution of the SageMath program ‘stream27.sage’, the output shown in Figure 3 will be displayed on the terminal.

As we wrap up this article, I can say with some satisfaction that we have covered as much theory and code as possible within the few pages available in a popular magazine like *OSFY* to discuss symmetric-key encryption. However, note that we have only seen the tip of the iceberg. To gain a better understanding of the implementation details of DES, Triple DES, AES, etc, I once again recommend the classic textbook ‘Cryptography and Network Security: Principles and Practice’ by William Stallings.

In the next article in this series, we will delve into public-key cryptography—a concept that is surprising, counter-intuitive, and fundamentally at odds with the principles of encryption that have evolved over two millennia. Stay tuned! 

 By: Dr Deepu Benson

The author is assistant professor at Chinmaya Vishwa Vidyapeeth (deemed to be university), Ernakulam, Kerala. He is a free software enthusiast and is interested in theoretical computer science.